ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KINH TẾ - LUẬT

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF ECONOMICS AND LAW

DATA MINING

Lecture 4: Classification



Lecture 4 outline

Section	What we cover
4.1.	Basic concepts of classification and prediction: the two-step process, decision tree induction, attribute selection, pruning
4.2.	Bayesian classification: Bayes' theorem, naive Bayes, Bayesian belief networks
4.3.	Classification by backpropagation: neural networks, forward and backward pass, gradient descent
4.4.	Classification based on rules and association rules, plus lazy learners (k-NN) and support vector machines
4.5.	Prediction: linear, multiple, non-linear and other regression models
4.6.	Classifier accuracy: confusion matrix, metrics, hold-out and cross-validation, ROC, ensembles, imbalanced data
Lab.	Lab 4 in Google Colab on saigon_bean_customers.csv: will this customer come back in September?

Lecture 4 roadmap and outcome

1	State a business question as a classification or a prediction task, name the label and the features, and say why the two-step process needs a separate test set	CLO4
2	Build a decision tree and a naive Bayes classifier by hand on a small credit table, and read the tree that scikit-learn grows on real customer data	CLO4
3	Explain how a neural network learns by backpropagation, how rules and k-NN classify, and fit a regression line for numeric prediction	CLO4
4	Read a confusion matrix, compute precision, recall, F1 and AUC, and defend the metric you chose for an unbalanced business problem	CLO4
5	Run a complete classification experiment in Colab with cross-validation and an ensemble, and report the result to a manager	CLO5

4.1

Basic Concepts of Classification and Prediction

4.1 Basic Concepts

What classification is, and what it is not

Classification

- Learn a model (a function) from data whose class label is already known
- Use the model to predict the categorical label of new, unseen objects
- **Output is a category: approve/refuse, default/no default, churn/stay, AAA/AA/A**
- Also called supervised learning: the label supervises the learning

Numeric prediction

- Same idea, but the target is a continuous number
- Predict next month's spend, the price of a house, the loss on a loan
- **Output is a number: 1,250,000 VND, 4.2 billion VND, 12.5%**
- Regression is the workhorse; section 4.5 covers it

```
age          income   collateral  credit_rating  | loan_approved     <- label (what we predict)
youth        high     no          fair           | no
middle_aged  high     no          fair           | yes
senior       medium   no          fair           | yes                <- one training tuple
```

The word "prediction" in the syllabus covers both. When the answer is a name, it is classification; when the answer is an amount, it is numeric prediction. Ask which one before choosing a method.

4.1 Basic Concepts

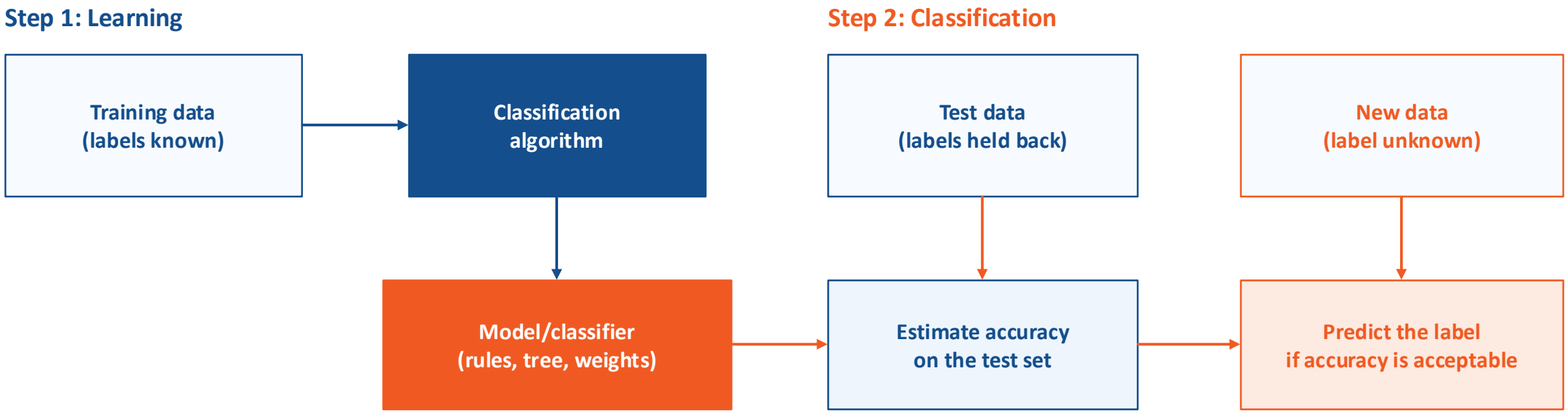
Three questions a bank can ask of the same data

Classification	Numeric prediction	Clustering
Will this applicant repay?	*How much will this customer spend next month?*	*What kinds of customers do we have?*
• Label: repaid/defaulted, known for past loans • Model: tree, Bayes, network, rules, k-NN, SVM • Output for a new file: a class, often with a probability • Chapter 4	• Target: a number known for past months • Model: linear or non-linear regression • Output: an amount in VND, with an error band • Chapter 4, section 4.5	• No label at all • Model: k-means, hierarchical, DBSCAN • Output: groups the analyst must name afterwards • Chapter 5

Same 4,777 customers, three different questions. The question decides the method, and the label (or its absence) decides whether the method is supervised.

4.1 Basic Concepts

The two-step process: learn, then classify




```
IF age = youth AND collateral = no        THEN loan_approved = no
IF age = middle_aged                      THEN loan_approved = yes
IF age = senior AND credit_rating = fair  THEN loan_approved = yes
# one possible model learned from the 14 credit applicants
```

Never measure accuracy on the data the model was trained on. It has already seen the answers, so the score would only show how well it memorised them.

4.1 Basic Concepts

The three ingredients, and the kinds of classification problem

Training set	Objects whose label is already known: the 14 past applications
Features (attributes)	The inputs: age, income, collateral, credit rating. Chapter 2 told us their types
Class label	The categorical attribute we want to predict: loan_approved

Kinds of classification problem

- Binary: two classes. Fraud/genuine transaction, churn/stay, default/repay
- Multi-class: one of several. Credit grade AAA to D, customer segment, product category
- Multi-label: several at once. A news article tagged both "economics" and "politics"

Saigon Bean question for the lab: label = returned_sept (1 or 0), features = everything we can compute from June to August behaviour. It is binary, and 47% of customers returned, so the classes are close to balanced.

Two ways to build the model

- Eager learners build the model before seeing any new object: decision trees, naive Bayes, neural networks, rules, SVM
- Lazy learners keep the training data and only compute when a new object arrives: k-nearest neighbours, case-based reasoning (section 4.4)

4.1 Basic Concepts

The 14 credit applicants (training set D)

id	age	income	collateral	credit_rating	loan_approved
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Read it as a bank officer

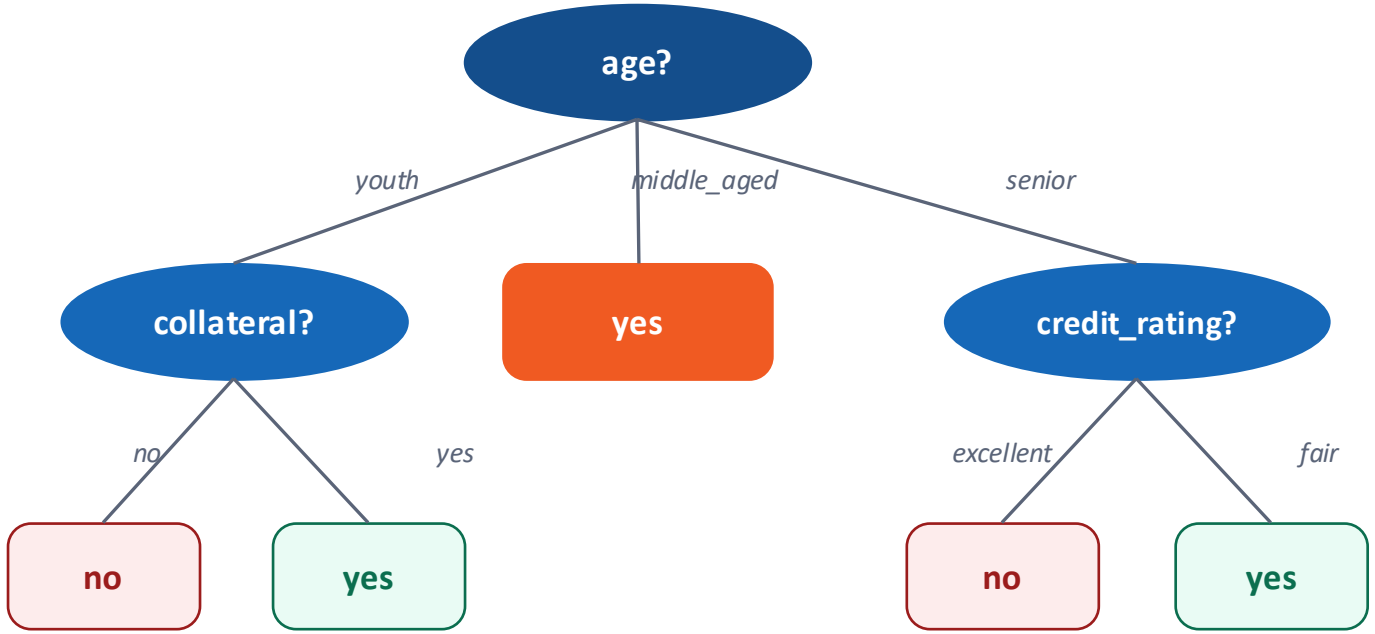
- 9 of 14 applications were approved, 5 refused
- All 4 middle-aged applicants were approved
- Youth: approved only when collateral = yes
- Senior: refused only when credit rating is excellent (odd, but it is the data)

What we will do with it

- Section 4.1: grow a decision tree by information gain and Gini
- Section 4.2: compute naive Bayes for a new applicant X = (youth, medium, collateral yes, fair)
- Section 4.4: turn the tree into IF-THEN rules

4.1 Decision Tree Induction

Anatomy of a decision tree



Four parts

- Root node: the first test, chosen on the whole training set (age)
- Internal node: a test on one attribute (collateral?, credit_rating?)
- Branch: one outcome of the test (youth, middle_aged, senior)
- Leaf: a class label. No further split (yes/no)

A path from the root to a leaf is a rule: IF age = youth AND collateral = yes THEN approve. That is why bank supervisors like trees: the decision can be explained to the customer and to the regulator.

Why is income not in the tree? Its information gain is the smallest (0.029), so the algorithm never needed it.

4.1 Decision Tree Induction

Growing the tree: top-down, greedy, recursive partitioning

1. Start at the root with all training tuples D
2. If all tuples in D belong to one class, make a leaf with that class and stop
3. If no attributes are left, make a leaf with the majority class (majority voting)
4. Otherwise choose the attribute that best separates the classes (attribute selection measure)
5. Split D into one subset per outcome of that attribute, and drop the attribute
6. Repeat from step 2 on every subset. Empty subset: leaf with the majority class of the parent

Three ways to split (Fig. 8.4)

- Discrete attribute: one branch per value (age: youth/middle_aged/senior)
- Continuous attribute: two branches, A <= split_point and A > split_point (income <= 42,000)
- Discrete, binary tree (CART): A in S_A or not (income in {low, medium})

Three families

- ID3 (Quinlan): information gain
- C4.5: gain ratio, handles continuous values and missing values
- CART: Gini index, always binary splits, also grows regression trees

4.1 Attribute Selection Measures

Information gain (ID3): how impure is D, and how much does a split clean it up?

Expected information (entropy) needed to classify a tuple in D

$$Info(D) = -\Sigma\ p_i \cdot log_2(p_i) \qquad (i = 1 \ldots m \text{ classes})$$

Information still needed after splitting D by attribute A

$$Info_A(D) = \Sigma\ (|D_j|/|D|) \cdot Info(D_j) \qquad (j = 1 \ldots v)$$

Information gain: choose the attribute with the largest gain

$$Gain(A) = Info(D) - Info_A(D)$$

On the credit table: Info(D)

- 9 approved, 5 refused, so p_yes = 9/14, p_no = 5/14

Info(D) = $-(9/14)\ log_2(9/14) - (5/14)\ log_2(5/14)$

= 0.940 bits

- Compare: a table with 7 yes and 7 no would give 1.000; a table with 14 yes gives 0.000
- The question for each attribute: after I know its value, how many bits are still needed?

Economic reading: entropy is uncertainty about the outcome. An attribute with high gain removes much of that uncertainty, the way a strong signal sharpens a forecast.

4.1 Attribute Selection Measures

Worked example: which attribute becomes the root?

Split by age

youth (5): 2 yes, 3 no → Info = 0.971

middle_aged (4): 4 yes, 0 no → Info = 0.000

senior (5): 3 yes, 2 no → Info = 0.971

Info_age(D) = (5/14)(0.971) + (4/14)(0) + (5/14)(0.971) = 0.694

Gain(age) = 0.940 − 0.694 = 0.246

Attribute	Info_A(D)	Gain(A)
age	0.694	0.246
collateral	0.788	0.151
credit_rating	0.892	0.048
income	0.911	0.029

age wins, so age is the root. Its middle_aged branch is already pure (4 yes) and becomes a leaf at once.

Then recurse on each branch with the remaining attributes

- youth subset (5 tuples): collateral gives Gain = 0.971, a perfect split → collateral? node
- senior subset (5 tuples): credit_rating gives a perfect split → credit_rating? node. All leaves are pure: the tree is complete

4.1 Attribute Selection Measures

Gain ratio (C4.5) and Gini index (CART)

Gain ratio: penalise attributes with many outcomes

$$SplitInfo_A(D) = -\Sigma\,(|D_j|/|D|)\cdot \log_2(|D_j|/|D|)$$

$$GainRatio(A) = Gain(A)/SplitInfo_A(D)$$

Gini index: impurity without logarithms, binary splits only

$$Gini(D) = 1 - \Sigma\, p_i^2$$

$$Gini_A(D) = (|D_1|/|D|)\cdot Gini(D_1) + (|D_2|/|D|)\cdot Gini(D_2)$$

Gain ratio on the credit table

SplitInfo_income(D) = $-(4/14)\log_2(4/14) - (6/14)\log_2(6/14) - (4/14)\log_2(4/14) = 1.557$

GainRatio(income) = 0.029/1.557 = 0.019

age: 0.246/1.577 = 0.156; collateral: 0.151/1.000 = 0.151

Same root (age); collateral now nearly ties. Real data: a 30-district attribute is heavily penalised, as it should be

Gini on the credit table

Gini(D) = $1 - (9/14)^2 - (5/14)^2 = 0.459$

income in {low, medium} vs {high}: (10/14)(0.480)+(4/14)(0.375) = 0.443, ΔGini = 0.016 (best income split)

age in {youth, senior} vs {middle_aged}: 0.357, ΔGini = 0.102

collateral: 0.367, ΔGini = 0.092; credit_rating: 0.429, ΔGini = 0.031. CART also picks age

4.1 Attribute Selection Measures

Comparing the three measures, other measures, and enhancements

Measure	Algorithm	Known bias
Information gain	ID3	Prefers attributes with many values (customer_id would win)
Gain ratio	C4.5	Prefers unbalanced splits where one partition is much smaller
Gini index	CART	Prefers many-valued attributes and equal-sized, pure partitions

Other measures you will meet in software

- CHAID: split chosen by a χ^2 test of independence (SPSS, marketing segmentation trees)
- C-SEP and the G-statistic (close to χ^2): better than gain and Gini in some cases
- MDL (minimum description length): the best tree is the one that needs the fewest bits to encode the tree plus its exceptions
- Multivariate splits: CART can split on a linear combination of attributes
- **Verdict of the textbook: most give good results, none is significantly superior**

Enhancements to basic induction

- Continuous attributes: sort the values, try the midpoints between neighbours as split points, keep the one with minimum expected information
- Missing values: assign the most common value, or spread the tuple over branches with probabilities
- Attribute construction: build new attributes from sparse ones (recency, frequency and monetary are constructed attributes)
- Discretise dynamically: define intervals of a continuous attribute during induction

4.1 Decision Tree Induction

Classification in large databases: RainForest and BOAT

Why scalability is a question

- Classic ID3 and C4.5 assume the training set fits in memory; a bank has millions of accounts
- Goal: build the same tree with few scans of the database and a memory footprint that depends on attributes, not on rows
- SLIQ and SPRINT (1996) sort once per attribute; RainForest (1998) and BOAT (1999) go further

RainForest: the AVC framework

- AVC-set of attribute X at a node: for each value of X, the count of each class label (Attribute, Value, Class)
- AVC-group of a node: the AVC-sets of all predictor attributes at that node; enough to compute gain, gain ratio or Gini
- Separates the scalability question from the split criterion; the tree is identical to the in-memory one

AVC-sets of the credit table at the root node

age	yes	no
youth	2	3
middle_aged	4	0
senior	3	2

collateral	yes	no
yes	6	1
no	3	4

income	yes	no
high	2	2
medium	4	2
low	3	1

credit_rating	yes	no
fair	6	2
excellent	3	3

BOAT (bootstrapped optimistic tree)

- Bootstrap samples that fit in memory, one tree each, combined into a tree T' close to the full-data tree
- Two scans of the database; incremental

4.1 Decision Tree Induction

Overfitting, and the two ways to prune

What goes wrong

- A tree grown to purity learns the noise: leaves with one or two tuples
- It fits the training set perfectly and the test set poorly
- Bank analogy: a credit rule with 200 clauses fits last year's files and fails on this year's

Prepruning (stop early)

- Stop splitting when a node is small, the gain is below a threshold, or a maximum depth is reached
- scikit-learn: max_depth, min_samples_leaf, min_impurity_decrease
- Risk: stopping too soon and missing a good split further down

Postpruning (grow, cut)

- Grow the full tree, then replace subtrees by leaves when that does not hurt error on a pruning set
- C4.5 pessimistic pruning; CART cost-complexity pruning (ccp_alpha)
- Usually the better tree, at the cost of computing the full one first

Tree on saigon_bean_customers.csv	Training accuracy	Test accuracy (30% hold-out)	Verdict
No depth limit (grown to purity)	100.0%	63.8%	Overfits: memorised the training set
max_depth = 4	72.1%	69.4%	Generalises: numbers are close

Rule of thumb: when training accuracy is far above test accuracy, the model is too complex for the data. Prune, or get more data.

4.1 Decision Tree Induction

Reading the tree scikit-learn grows on the customer file (depth 3)

```
from sklearn.tree import DecisionTreeClassifier, export_text
tree = DecisionTreeClassifier(max_depth=3, random_state=42)
tree.fit(X_train, y_train)
print(export_text(tree, feature_names=list(X.columns)))

|--- recency_days <= 20.5
|   |--- frequency <= 12.5
|   |   |--- has_loyalty_card <= 0.5   -> class: 0 (no)
|   |   |--- has_loyalty_card >  0.5   -> class: 1 (returns)
|   |--- frequency >  12.5             -> class: 1 (returns)
|--- recency_days >  20.5
|   |--- recency_days <= 48.5
|   |   |--- frequency <= 13.5         -> class: 0 (no)
|   |   |--- frequency >  13.5         -> class: 1 (returns)
|   |--- recency_days >  48.5          -> class: 0 (no)
```

How to read it

- Root = recency_days: the number of days since the last order is the single most informative feature
- Each split on a numeric attribute is a binary <=/> test at a midpoint
- A leaf reports the majority class of the customers that reach it

What the manager hears

- "A customer who bought in the last three weeks and has a loyalty card will very likely be back"
- "After seven weeks of silence, nothing else we know about the customer changes the answer: they are gone"
- Rules a marketing team can act on, straight from the tree

Practice 1

1. A telecom company wants to know which prepaid subscribers will top up again within 30 days. Write the task as a classification problem: name the label, list six features you could compute from call and top-up history, and say which of them is ordinal.

2. On the credit table, compute Info_collateral(D) and Gain(collateral) by hand, showing the two partitions and their entropies. Check that you obtain 0.788 and 0.151. Then compute Gini for the split credit_rating = fair versus excellent.

3. A colleague grows a decision tree on 3,343 customers with no depth limit and reports 100% accuracy. Explain in three sentences why the number is not evidence that the model is good, and what you would ask for instead.

4.2

Bayesian Classification Methods

Bayes' theorem, the naive Bayesian classifier, and Bayesian belief networks

4.2 Bayesian Classification

Bayes' theorem, in the bank's words

$$P(H \mid X) = P(X \mid H) \cdot P(H)/P(X)$$

Term	Name	In the bank
P(H)	Prior probability of class H	Share of past applications that were approved: 9/14 = 0.643
P(X | H)	Likelihood of the evidence given the class	How often approved applicants looked like X (youth, medium income, …)
P(X)	Evidence, a normalising constant	How often any applicant looks like X; the same for every class
P(H | X)	Posterior probability	What we want: the chance that this applicant X is approved

The decision rule

- Compute the posterior for every class C_i
- **Predict the class with the largest P(C_i | X): the maximum a posteriori (MAP) hypothesis**
- Since P(X) is common to all classes, maximise P(X | C_i) · P(C_i) instead

Why economists like it

- Prior = base rate: fraud at 0.5% is a strong prior to overcome
- Likelihood = how typical this evidence is for each class
- Posterior = belief updated by data, as in Bayesian econometrics

4.2 Bayesian Classification

The naive Bayesian classifier: one strong, useful assumption

The problem

- X = (x_1, x_2, ..., x_n) has n attributes
- P(X | C_i) needs the frequency of the whole combination (youth AND medium AND yes AND fair) among approved applicants
- With 14 rows most combinations never occur; with 4,777 customers and 11 features, the same

The naive assumption

- **Given the class, the attributes are independent of one another (class-conditional independence)**
- Then the joint probability is the product of the marginals, each estimated by a simple count
- "Naive" because income and collateral are surely related; we pretend they are not

$$P(X \mid C_i) = \Pi\, P(x_k \mid C_i) \quad \rightarrow \quad predict \;\; argmax_i \;\; P(C_i) \cdot \Pi\, P(x_k \mid C_i)$$

How each factor is estimated

- Categorical: P(x_k | C_i) = class-C_i tuples with value x_k, divided by all class-C_i tuples
- Continuous: a Gaussian within the class, from its mean and standard deviation (Gaussian naive Bayes, used in the lab)

4.2 Bayesian Classification

Worked example: new applicant X = (youth, medium, collateral yes, fair)

Step 1: priors from the 14 rows

P(approved = yes) = 9/14 = 0.643 **P(approved = no) = 5/14 = 0.357**

Conditional probability	approved = yes	approved = no
P(age = youth | C)	2/9 = 0.222	3/5 = 0.600
P(income = medium | C)	4/9 = 0.444	2/5 = 0.400
P(collateral = yes | C)	6/9 = 0.667	1/5 = 0.200
P(credit_rating = fair | C)	6/9 = 0.667	2/5 = 0.400
P(X | C) = product	0.044	0.019

Step 3: multiply by the prior and compare

P(X | yes) · P(yes) = 0.044 × 0.643 = 0.028

P(X | no) · P(no) = 0.019 × 0.357 = 0.007

0.028 > 0.007, so naive Bayes predicts loan_approved = yes

Normalised: P(yes | X) = 0.028/(0.028 + 0.007) = 0.80

Same answer as the tree

- The tree sent X to the youth → collateral = yes leaf: yes
- Bayes weighed all four attributes at once and produced a probability as well as a label
- That probability is what a bank uses to set a threshold or a price for risk

4.2 Bayesian Classification

Zero probabilities, the Laplacian correction, and the three variants

The zero-frequency problem

- Suppose no refused applicant ever had income = low: P(low | no) = 0/5 = 0
- Then P(X | no) = 0 for every X with low income, whatever the other attributes say
- **Laplacian correction: add 1 to each count. With 1,000 tuples and counts 0, 990, 10 → 1/1003, 991/1003, 11/1003**
- Close to the originals, but no zero. scikit-learn: alpha = 1 (var_smoothing for Gaussian)

```
from sklearn.naive_bayes import GaussianNB
nb = GaussianNB().fit(X_train, y_train)
nb.predict_proba(X_test[:1])
# array([[0.31, 0.69]])  -> P(no), P(returns)
# Lab: accuracy 0.656, AUC 0.748 on the hold-out set
```

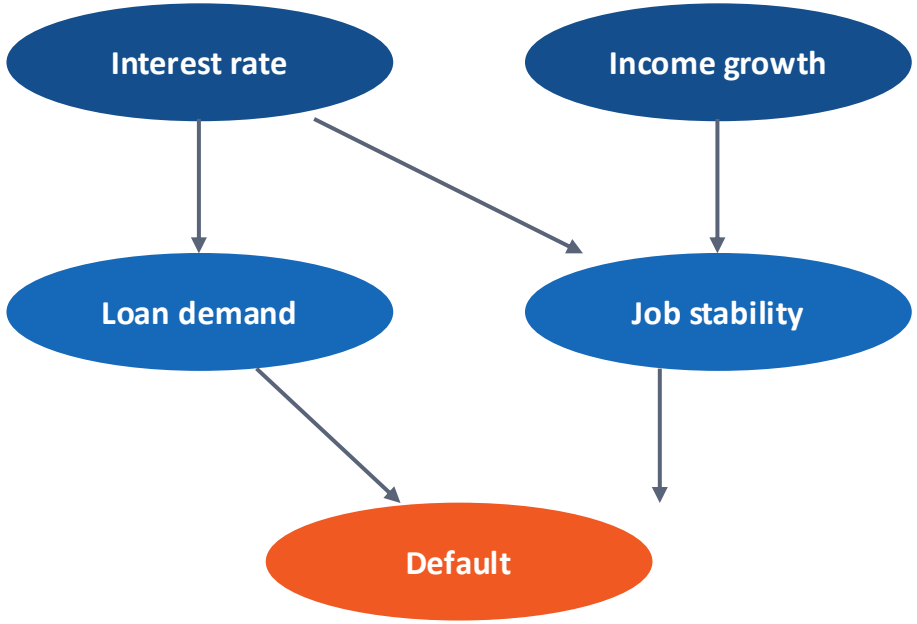
Variant	Attribute type	Business use
`GaussianNB`	Continuous (recency, spend)	Customer return, credit scoring
`MultinomialNB`	Counts (word frequencies)	Complaints, news, reviews
`BernoulliNB`	Binary presence/absence	Products in the basket, channels used

Strengths and limits

- Fast, needs little data, handles many attributes, gives calibrated-looking probabilities
- Text classification and spam filters: the textbook case
- Limit: dependent attributes are double-counted. Answer: Bayesian belief networks (next slide)

4.2 Bayesian Classification

Bayesian belief networks: when independence is too naive



P(Default | LD, JS)	high, stable	high, unstable	low, stable	low, unstable
yes	0.08	0.35	0.04	0.20
no	0.92	0.65	0.96	0.80

What a belief network is

- A directed acyclic graph: nodes are variables, arcs are direct dependencies (parent → child)
- Each node has a conditional probability table (CPT) given its parents; the table on the left is one CPT
- Any node can be the class label; the network can answer with missing evidence

Learning and using it

- Structure known, data complete: fill the CPTs by counting (like naive Bayes)
- Structure known, some values hidden: gradient descent or EM on the CPT entries
- Structure unknown: search over graphs; or let a domain expert draw the arrows (illustrative CPT values on the left)

Practice 2

1. Using the credit table, classify a new applicant X = (middle_aged, low income, collateral no, credit rating excellent) with naive Bayes. Show the priors, the eight conditional probabilities and the two products. Which class wins?

2. One of your conditional probabilities in task 1 is zero. Recompute the affected class with the Laplacian correction (add 1 to each count, and the number of values of that attribute to the denominator). Does the decision change?

3. A payment provider sees fraud in 0.5% of transactions. A naive Bayes model reports P(X | fraud) = 0.30 and P(X | genuine) = 0.002 for a suspicious transaction. Compute both posteriors up to the constant and decide. Then say what happens to the decision if the fraud rate rises to 5%.

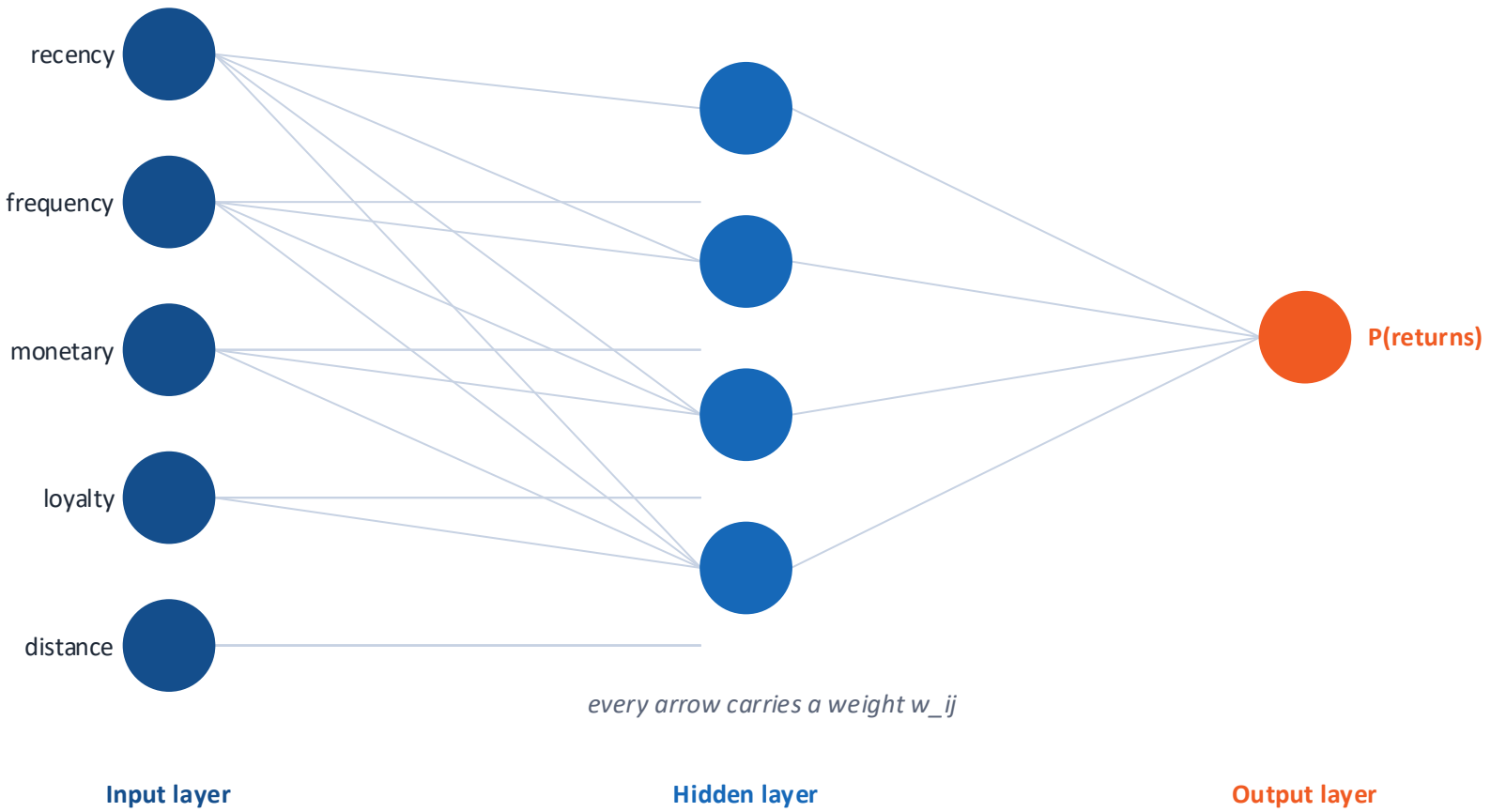
4.3

Classification by Backpropagation

Multilayer feed-forward neural networks and how they learn from their errors

4.3 Classification by Backpropagation

A multilayer feed-forward network on the customer problem



The parts

- Input units: one per feature, after scaling to [0, 1]
- Hidden units: weighted sums pushed through an activation function
- Output unit: one for a binary label, one per class otherwise
- Weights w_ij and biases θ_j: what the network learns

Feed-forward

- Signals move only from input to output; no cycles
- Fully connected: every unit feeds every unit of the next layer
- Topology (layers, units) is chosen by the analyst, often by trial and error

4.3 Classification by Backpropagation

Inside one unit, and why logistic regression is a one-unit network

Net input of unit j: weighted sum of its inputs plus a bias

$$I_j = \Sigma_i \; w_{ij} \cdot O_i \; + \; \theta_j$$

Output of unit j: the activation function applied to the net input

$$O_j = 1/(1 + e^{(-I_j)}) \qquad (sigmoid/logistic)$$

Activation	Formula	Where it is used
Sigmoid	`1/(1 + e^(-x))`	Output unit for a binary label; classic hidden units
tanh	`(e^x - e^(-x))/(e^x + e^(-x))`	Hidden units, centred at 0
ReLU	`max(0, x)`	Hidden units in modern deep networks; fast
Softmax	`e^(x_k)/Σ e^(x_l)`	Output layer for multi-class problems

Logistic regression, seen again

$$P(y = 1 \mid x) = 1/(1 + e^{(-(\beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n))})$$

- That is one output unit with a sigmoid and no hidden layer
- Its boundary is a straight line (hyperplane); adding hidden units lets the boundary curve

Before training: prepare the inputs

- Scale every numeric feature to [0, 1] or standardise; otherwise monetary_vnd (millions) swamps weekday_share (0 to 1)
- One-hot encode main_channel and main_store_type
- Initialise weights to small random numbers, for example in [−1, 1]

4.3 Classification by Backpropagation

The learning loop: forward pass, error, backward pass, update

1. **Forward propagation:** Feed one training tuple. Each unit computes I_j and O_j, layer by layer, until the output O_k appears
2. **Measure the error:** Compare O_k with the true label T_k. Loss = squared error or cross-entropy, summed over the tuple
3. **Backpropagate the error:** Compute Err_j for the output units, then for the hidden units using the errors of the layer they feed
4. **Update weights and biases:** Move every weight a small step against the error gradient (gradient descent), scaled by the learning rate l

Repeat over all tuples (one epoch), then start again, until the weights stop changing or the error on a validation set stops falling.

The update rule (sigmoid units, eq. 9.4 to 9.9)

$$\text{Output unit: } Err_j = O_j(1 - O_j)(T_j - O_j)$$

$$\text{Hidden unit: } Err_j = O_j(1 - O_j)\sum_k Err_k \cdot w_{jk}$$

$$\Delta w_{ij} = (l) \cdot Err_j \cdot O_i, \quad \Delta\theta_j = (l) \cdot Err_j$$

Three knobs

- Learning rate l (0 to 1): too large and the search oscillates, too small and it crawls. Rule of thumb 1/t at iteration t
- Epochs: one full pass over the training set; hundreds are common
- Stopping: when all Δw are tiny, or a fixed number of epochs, or early stopping on validation loss

4.3 Classification by Backpropagation

Strengths, limits, and the network in the lab

Strengths

- Tolerant of noisy data, and can classify patterns it has not been trained on
- Learns non-linear boundaries: interaction of recency and frequency without hand-made features
- Same machinery scales to images, text, time series (deep learning)

Limits

- Long training, many parameters to set by hand (topology, learning rate)
- A black box: weights carry no business meaning; hard to explain a refusal
- Needs scaled numeric inputs and, usually, more data than a tree

Opening the box

- Sensitivity analysis: vary one input, watch the output
- Rule extraction: cluster hidden-unit activations, derive IF-THEN rules
- SHAP values and partial dependence in modern practice

```
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
mlp = make_pipeline(StandardScaler(),
        MLPClassifier(hidden_layer_sizes=(16, 8), max_iter=800,
random_state=42))
mlp.fit(X_train, y_train)    # backpropagation, up to 800 epochs
```

Model on the customer file (test set)	Accuracy	F1	AUC
Logistic regression (no hidden layer)	0.734	0.711	0.803
MLP 16-8 (backpropagation)	0.714	0.688	0.778
Decision tree, depth 4	0.694	0.622	0.769

Practice 3

1. A unit has inputs $O_1 = 1$, $O_2 = 0$, $O_3 = 1$ with weights $w_1 = 0.2$, $w_2 = 0.4$, $w_3 = -0.5$ and bias $\theta = -0.4$. Compute the net input I and the sigmoid output O. Then compute the output-unit error Err if the true label is $T = 1$.

2. With learning rate $l = 0.9$, compute the new value of w_1 and of θ after one backpropagation step using the error from task 1. Explain in one sentence why w_2 does not change.

3. A fintech lender obtains AUC 0.81 with a neural network and 0.80 with logistic regression on the same loan data. The regulator requires a written reason for every refusal. Which model would you deploy, and what would you say to the data scientist who prefers the network?

4.4

Rule-Based and Associative Classification

IF-THEN rules, sequential covering, association rules for classification, k-NN and SVM

4.4 Rule-Based Classification

IF-THEN rules: coverage, accuracy, and rules from a tree

A rule R: IF condition (antecedent) THEN class (consequent)

```
R1: IF age = youth AND collateral = yes THEN approved = yes
```

$$coverage(R) = n_covers/|D|$$

$$accuracy(R) = n_correct/n_covers$$

On the credit table

R1 covers the 2 youth applicants with collateral (ids 9, 11): coverage = 2/14 = 14.3%

Both were approved: accuracy = 2/2 = 100%

Rules can also be pruned: drop a condition if the rule's estimated accuracy on a pruning set does not fall. C4.5 does this to the tree rules, and the result is often shorter than the tree.

```
Rules extracted from the tree of section 4.1
R1: IF age=youth AND collateral=no          THEN no
R2: IF age=youth AND collateral=yes         THEN yes
R3: IF age=middle_aged                      THEN yes
R4: IF age=senior AND rating=excellent      THEN no
R5: IF age=senior AND rating=fair           THEN yes
```

Properties of a rule set

- Mutually exclusive: no tuple triggers two rules (true for tree rules, false in general)
- Exhaustive: every tuple triggers some rule; otherwise add a default rule for the majority class
- Conflicts: size ordering (longest antecedent wins) or rule ordering by accuracy, or class-based voting

4.4 Rule-Based Classification

Learning rules directly: sequential covering (Fig. 8.10)

1. Pick a class c (say approved = no)
2. Learn one rule that covers many tuples of c and few of other classes: start empty, add the best condition each step
3. Remove the tuples the rule covers from D
4. Repeat until no tuples of c remain, or the rule quality falls below a threshold
5. Move to the next class; the last class becomes the default rule

Pruning a rule uses a separate pruning set: remove a condition if FOIL_Prune = (pos – neg)/(pos + neg) rises.

Rule quality: how to choose the next condition

$$FOIL_Gain = pos' \cdot [\log_2(pos'/(pos'+neg')) - \log_2(pos/(pos+neg))]$$

Read it

- pos, neg: tuples covered by the current rule; pos', neg': after adding the condition
- Favours conditions that keep many positives and gain accuracy
- Accuracy alone would prefer a rule covering one tuple

Algorithms

- AQ, CN2, RIPPER (repeated incremental pruning): the classic family
- Tree rules vs covering rules: the tree partitions once; covering may let rules overlap

4.4 Associative Classification

Association rules with the class on the right-hand side

The idea

- Chapter 3 mined itemset → itemset. Now mine feature values → class label
- **Class association rule: (recency ≤ 20) ∧ (loyalty = yes) ⇒ returned = 1 [support 21%, confidence 88%]**
- Support and confidence are computed exactly as in Chapter 3; only the consequent is constrained

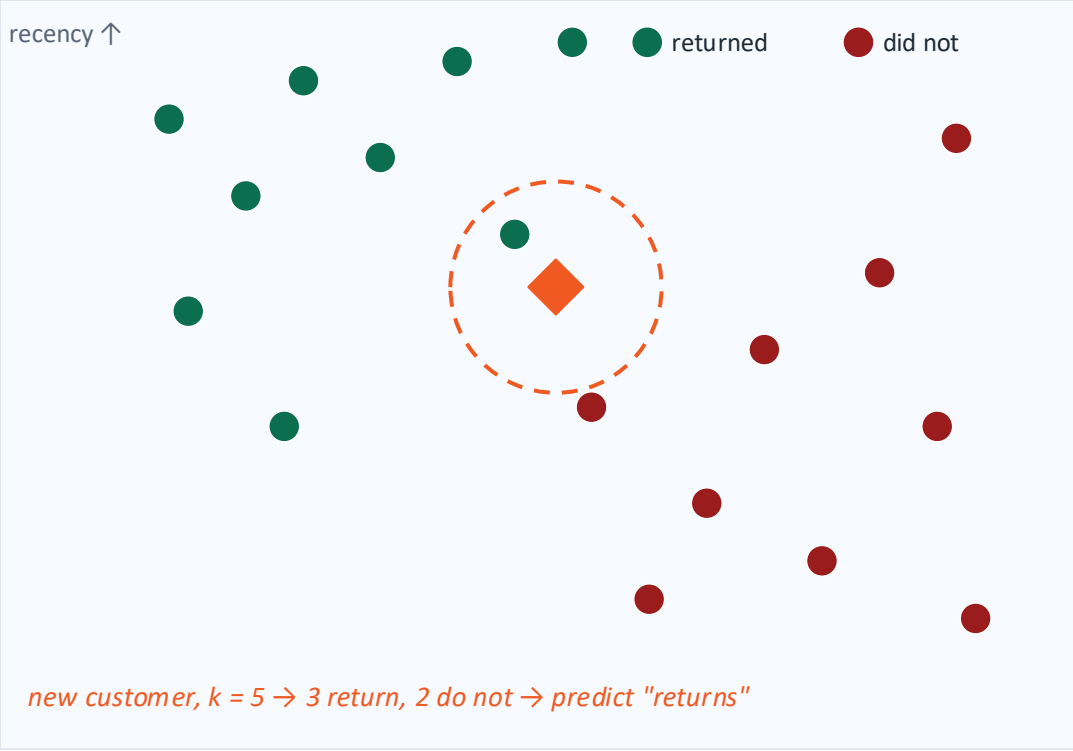
Why bother, when we have trees?

- Trees test one attribute at a time; association rules find combinations directly
- Often more accurate than C4.5 on the same data (textbook), and every rule reads as a policy
- The mining step is expensive on many attributes; discretise numeric ones first

Method	How it selects rules	How it classifies a new tuple
CBA (Classification Based on Associations)	Mine strong class rules, order by confidence then support, prune	First rule whose antecedent matches; default rule otherwise
CMAR (Multiple Association Rules)	Stores rules in a tree, prunes by confidence and χ^2	All matching rules vote, grouped by class, weighted χ^2
CPAR (Predictive Association Rules)	Greedy rule generation like FOIL, reuses covered tuples	Best k rules per class, averaged expected accuracy

4.4 Lazy Learners

k-nearest neighbours: classify by looking at similar customers



The algorithm, in four lines

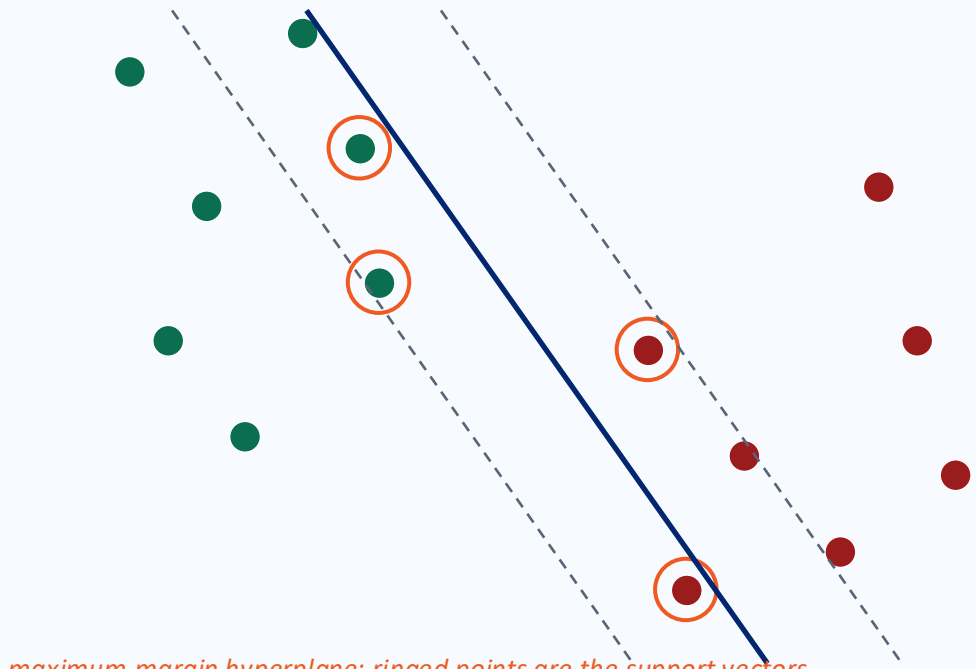
- Store the training tuples. Choose k (odd: 3, 5, 15)
- For a new tuple, compute its distance to every stored tuple (Euclidean after scaling; Chapter 2 measures for mixed types)
- Take the k closest; predict the majority class (or the average value for numeric prediction)
- Optional: weight each vote by $1/d^2$

Choosing k, and the costs

- k = 1 follows the noise (overfits); large k blurs the boundary (underfits). Tune k on a validation set: the lab tries 1 to 40 and lands on k = 39
- Slow at prediction time: every query scans all 3,343 training customers. Index structures and editing help
- Lab result with k = 39 on scaled features: accuracy 0.679, AUC 0.762

4.4 Support Vector Machines

SVM: the widest street between two classes, and the kernel trick



maximum margin hyperplane; ringed points are the support vectors

Linearly separable case

- Many lines separate the two classes; SVM picks the one with the largest margin (distance to the nearest points of each class)
- The nearest points are the support vectors; the model is defined by them alone
- Solved as a constrained quadratic optimisation; a global optimum, no local minima

Non-linear case: the kernel trick

- Map the data to a higher-dimensional space where a hyperplane separates it. The map is never computed: a kernel K(x, y) replaces the dot product
- Kernels: linear, polynomial $(x \cdot y + 1)^h$, RBF $\exp(-\|x - y\|^2/2\sigma^2)$, sigmoid
- Strong on high-dimensional, small-sample data (text, genes); slow on very large tables. Parameter C trades margin for errors

4.4 Support Vector Machines

The maximum marginal hyperplane, and why SVM copes with many dimensions

Linearly separable data D = {(X_i, y_i)}, y_i ∈ {+1, −1}

Separating hyperplane: $W \cdot X + b = 0$ *(2-D:* $w_0 + w_1 x_1 + w_2 x_2 = 0$*)*

Margin sides: $H_1: W \cdot X + b \geq +1$ *for* $y_i = +1$, $H_2: W \cdot X + b \leq -1$ *for* $y_i = -1$

Maximise the margin $2/\|W\|$ *subject to* $y_i (W \cdot X_i + b) \geq 1$

How it is solved, and what comes out

- A convex quadratic programme with linear constraints: Lagrange multipliers, one per tuple; a global optimum
- Tuples on H_1 or H_2 are the support vectors; only they have non-zero multipliers
- Classify X by the sign of Σ y_i α_i X_i · X + b_0 over the support vectors: the rest of the data can be deleted

Why effective on high-dimensional data

- Model complexity depends on the number of support vectors, whatever the number of attributes
- The number of support vectors gives an upper bound on the expected error, independent of dimensionality
- Few support vectors → good generalisation even with thousands of features (text, genes)

	SVM	Neural network
Algorithm	Deterministic, batch QP	Non-deterministic, incremental
Foundation	Statistical learning theory	Weaker theory
Complex functions	Through kernels	Through hidden layers
Scaling	Hard on millions of rows	Epoch costs O(|D|·w)

4.4 Other Classification Methods

Case-based reasoning, genetic algorithms, rough sets and fuzzy sets

Case-based reasoning (CBR)

- A lazy learner over symbolic cases, not points in Euclidean space
- Retrieve similar cases, combine their solutions, adapt to the new problem
- Customer service diagnosis, legal rulings, insurance claims
- Challenge: a good similarity measure and case indexing

Genetic algorithms

- Rules encoded as bit strings (IF A_1 AND NOT A_2 THEN C_2 → 100)
- Fitness = accuracy on training data; keep the fittest, breed by crossover and mutation
- Stop when every rule passes a threshold
- Slow, but easily parallelised; used for feature selection and rule search

Rough sets

- A class C is bracketed by a lower approximation (certainly in C) and an upper approximation (cannot be excluded)
- Handles inconsistent, noisy data with equivalence classes
- Reducts: minimal attribute subsets, found via a discernibility matrix
- Feature reduction in credit and marketing data

Fuzzy sets

- Membership degrees in [0, 1] instead of hard cut-offs: income 49 million is 'medium' with 0.15 and 'high' with 0.96
- Every applicable rule votes with its truth value; votes are combined
- Reads like the language of a credit policy
- Fuzzy logic controllers, risk grading, recommender rules

4.4 Beyond Two Classes and Beyond Labelled Data

Multiclass strategies, semi-supervised, active and transfer learning

Multiclass classification with binary learners

- One-vs-all (OVA): m classifiers, class j against the rest; classifiers vote
- All-vs-all (AVA): m(m−1)/2 pairwise classifiers, majority vote; usually better than OVA but sensitive to single errors
- Error-correcting output codes: each class gets a codeword (7 bits for 4 classes) and one classifier is trained per bit. Pick the class with the smallest Hamming distance: output 0001010 is 1 bit from C4 = 0101010, so X → C4

Semi-supervised classification

- Uses a few labelled and many unlabelled tuples (most customer data is like this)
- Self-training: fit on labelled data, label the unlabelled, add the most confident, repeat; may reinforce its own errors
- Co-training: two classifiers on independent feature sets teach each other (transaction features vs app-usage features)

Active learning

- Labels cost money (a survey, a manual credit review): let the model choose which tuples to ask an oracle about
- Uncertainty sampling: query the tuples the model is least sure of; judged by a learning curve of accuracy against number of queries

Transfer learning

- Reuse knowledge from a source task (last year's churn model, another city) on a target task with little data
- Useful when the distribution drifts; instance re-weighting (TrAdaBoost) is the classic method

Practice 4

1. On the credit table, evaluate the rule IF credit_rating = excellent THEN loan_approved = no. Compute its coverage and accuracy. Then propose one extra condition that raises accuracy to 100%, and report the new coverage.

2. Three stored customers have (recency_days, monetary_vnd) = (5, 400,000), (40, 420,000), (8, 900,000) with labels return, no, return. A new customer is at (7, 410,000). Find the 1-NN prediction with raw Euclidean distance, then again after min-max scaling both features to [0, 1]. Explain the difference.

3. A bank wants a model it can turn into written credit policy that loan officers apply without a computer. Rank decision tree, rule-based classifier, naive Bayes, k-NN and SVM for this purpose, and justify the top and the bottom of your list.

4.5

Prediction: Regression Models

Linear, multiple, non-linear and other regression models for numeric prediction

4.5 Prediction

Linear regression: one predictor, least squares

Model and least-squares estimates

$$y = w_0 + w_1 x$$

$$w_1 = \Sigma (x_i - \bar{x})(y_i - \bar{y}) / \Sigma (x_i - \bar{x})^2, \quad w_0 = \bar{y} - w_1 \bar{x}$$

x: years	3	8	9	13	3	6	11	21	1	16
y: salary (M VND)	30	57	64	72	36	43	59	90	20	83

Compute

$\bar{x}$ = 9.1 years, $\bar{y}$ = 55.4 million

w_1 = 3.54, w_0 = 55.4 − 3.54 × 9.1 = 23.2

y = 23.2 + 3.54 x → 10 years: 58.6 million VND per month



Data-mining view: the fitted line is a predictor. Its quality is judged on new employees (a test set) by RMSE or MAE. Econometrics judges the same line by the t-statistic on w_1.

4.5 Prediction

Multiple, non-linear and other regression models

Multiple linear regression on sept_spend_vnd (returning customers)

$$y = w_0 + w_1 x_1 + w_2 x_2 + \ldots + w_n x_n$$

```
from sklearn.linear_model import LinearRegression
reg = LinearRegression().fit(X_r, y_r)
# sept_spend = -129,065 + 10,500*frequency + 2.9*avg_basket
#              + 13,097*has_loyalty_card - 104.5*recency_days
# R^2 = 0.805 on 2,231 returning customers
```

Read the coefficients as a manager

- One extra order in the summer is worth about 10,500 VND of September spend, other things equal
- A loyalty card adds about 13,000 VND; each day of silence removes about 100 VND
- $R^2 = 0.805$: the four features explain 80% of the variance of September spend

Non-linear regression

- Polynomial: $y = w_0 + w_1x + w_2x^2 + w_3x^3$. Set $x_1 = x$, $x_2 = x^2$, $x_3 = x^3$ and it is multiple linear regression again
- Any model linear in the parameters can be fitted by least squares after a transformation (log, square root)
- Truly non-linear models (exponential growth) need iterative solvers

Other models

- Logistic regression: a classifier that predicts P(class) with a sigmoid of a linear score
- Poisson regression: counts, such as orders per month
- Log-linear models: probabilities in multidimensional tables (data cubes)
- Regression trees and model trees: a tree whose leaves hold a number or a small linear model

4.5 Prediction

Measuring a predictor's error, and trees that predict numbers

Measure	Formula (test set, d tuples)	Reading
Mean absolute error (MAE)	$\Sigma \lvert y_i - \hat{y}_i \rvert / d$	Average miss, in VND; robust to outliers
Mean squared error (MSE)	$\Sigma (y_i - \hat{y}_i)^2 / d$	Punishes large misses; unit is VND²
Root mean squared error (RMSE)	$\sqrt{MSE}$	Back in VND; the usual headline number
Relative absolute error	$\Sigma \lvert y_i - \hat{y}_i \rvert / \Sigma \lvert y_i - \bar{y} \rvert$	Relative to always predicting the mean; < 1 beats the mean
Relative squared error	$\Sigma (y_i - \hat{y}_i)^2 / \Sigma (y_i - \bar{y})^2$	Its square root is the RRSE; 1 – RSE = R^2

Lab result: the September-spend regression has RMSE ≈ 52,000 VND and $R^2 = 0.81$ on the hold-out. Read it as: on a typical returning customer we miss by about fifty thousand dong, and we explain four fifths of the variation.

Regression trees and model trees

- Regression tree (CART, 1984): each leaf stores the average target value of the tuples that reach it
- Model tree (Quinlan, 1992): each leaf stores a small linear regression; more general
- Both beat linear regression when the data are not one straight line

Practice 5

1. Five stores have (seats, daily orders) = (20, 210), (40, 380), (70, 590), (24, 240), (55, 470). Fit $y = w_0 + w_1 x$ by least squares and predict daily orders for a planned 60-seat store. State one reason the prediction could be wrong for a store in a new district.

2. Using the regression on the previous slide, estimate September spend for a returning customer with 8 summer orders, an average basket of 60,000 VND, a loyalty card, and 10 days since the last order. Then say how much the estimate changes if the card is removed.

3. For each target, name the regression model you would fit and why: (a) whether a customer returns, (b) the number of complaints a store receives in a month, (c) house price given area, with a clear diminishing effect of area.

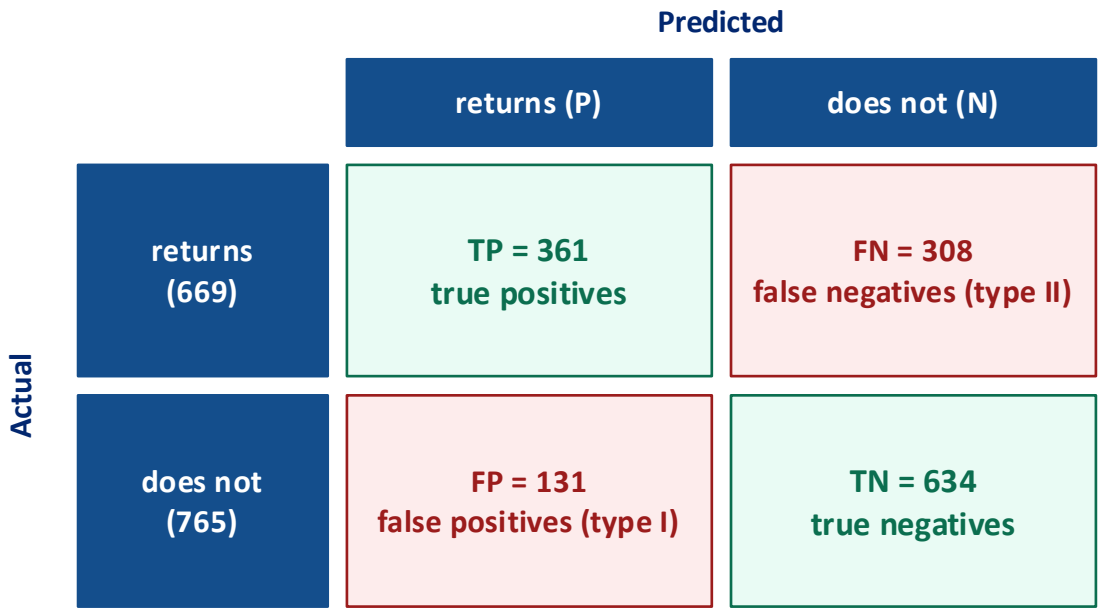
4.6

Classifier Accuracy

Estimating accuracy, choosing between models, and improving accuracy

4.6 Evaluating Classifiers

The confusion matrix: four kinds of outcome, two kinds of mistake



Decision tree (max_depth = 4), 1,434 test customers, positive class = returned in September

```
from sklearn.metrics import confusion_matrix
confusion_matrix(y_test, tree.predict(X_test))
# [[634 131]   <- rows: actual 0, actual 1 (sklearn order)
#  [308 361]]
```

Read it

- Rows are the truth, columns the model's answer; the diagonal is right, the rest is wrong
- P = TP + FN = 669 real returners; N = FP + TN = 765 who did not return
- The model found 361 of the 669 returners and wrongly flagged 131 of the 765 others

Two mistakes, two costs

- False positive: we predicted a return that did not happen. Cost: a retention offer wasted, or a loan granted to a defaulter
- False negative: we missed a real return. Cost: a customer we could have kept, or a good borrower refused
- The business decides which is worse. That choice picks the metric on the next slide

4.6 Evaluating Classifiers

From the matrix to the metrics (Fig. 8.13), computed on the tree

Measure	Formula	Tree, depth 4	Meaning for the marketing team
Accuracy	$(TP + TN)/(P + N)$	995/1434 = 0.694	Share of customers classified correctly
Error rate	$(FP + FN)/(P + N)$	0.306	1 – accuracy
Recall (sensitivity)	TP/P	361/669 = 0.540	Of the real returners, how many did we catch?
Specificity	TN/N	634/765 = 0.829	Of the non-returners, how many were left alone?
Precision	$TP/(TP + FP)$	361/492 = 0.734	Of the flagged customers, how many returned?
F1	$2 \cdot P \cdot R/(P + R)$	0.622	Punishes a weak precision or a weak recall
F_β	$(1+\beta^2) \cdot P \cdot R/(\beta^2 \cdot P + R)$	β = 2: 0.571	β > 1 favours recall; β < 1 favours precision

Accuracy paradox: a model that always says 'no' scores 0.534 here (recall 0). On a file with 0.5% fraud, 'never fraud' scores 99.5%.

Choose the metric from the cost: default screening wants recall; a small retention budget wants precision; a balanced report uses F1 and AUC.

4.6 Estimating Accuracy

Hold-out, cross-validation and bootstrap: where does the test set come from?

Hold-out

- Split once: 2/3 train, 1/3 test (the lab uses 70/30, stratified)
- Estimate is pessimistic (less training data) and depends on the split
- Random subsampling: repeat k times and average

k-fold cross-validation

- Cut D into k equal folds; train on k − 1, test on the remaining fold; rotate k times
- Every tuple is tested exactly once; accuracy = correct/|D| over all folds
- k = 10 is the norm; stratified keeps class shares in every fold; leave-one-out is k = |D|

Bootstrap (.632)

- Sample |D| tuples with replacement: 63.2% distinct tuples form the training set, the 36.8% never drawn form the test set
- Acc = Σ (0.632 · acc_test + 0.368 · acc_train)/k over k samples
- Good for small data sets; it is also the engine of bagging

```
from sklearn.model_selection import cross_val_score,
StratifiedKFold
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=1)
scores = cross_val_score(DecisionTreeClassifier(max_depth=4), X, y,
cv=cv)
scores.round(3)
# [0.728 0.713 0.676 0.728 0.713 0.695 0.738 0.730 0.673 0.698]
# mean 0.709, std 0.022
```

Comparing two models honestly

- Run both on the same 10 folds and compare mean accuracy with a paired t-test on the 10 differences
- If the confidence interval contains 0, the difference is noise. With std 0.022, a 1-point gap is not a decision

4.6 Model Selection

Is the difference real? Confidence intervals and the paired t-test

The question

- Two classifiers M_1 and M_2; 10-fold cross-validation gives mean error rates err(M_1) and err(M_2)
- These are estimates on a sample of future cases; the difference could be chance
- Null hypothesis H_0: M_1 and M_2 have the same error rate. Reject it, and the difference is statistically significant

Paired t-test on the same k folds, k − 1 = 9 d.f.

$$d_i = err(M_1)_i - err(M_2)_i, \quad t = \bar{d}/(\, s_d/\sqrt{k}\,)$$

$$s_d^2 = \Sigma\,(d_i - \bar{d})^2/(k-1); \quad \textit{two test sets: unpaired t with pooled variance}$$

Reject H_0 at significance 5% if $|t| > t_{0.025,9} = 2.262$ (t-table, symmetric, upper-tail column sig/2).

Fold	1	2	3	4	5	6	7	8	9	10
Tree d=4	.728	.713	.676	.728	.713	.695	.738	.730	.673	.698
Logistic	.749	.724	.695	.741	.745	.722	.770	.746	.719	.713
d_i	.021	.011	.019	.013	.032	.027	.032	.016	.046	.015

On the lab data

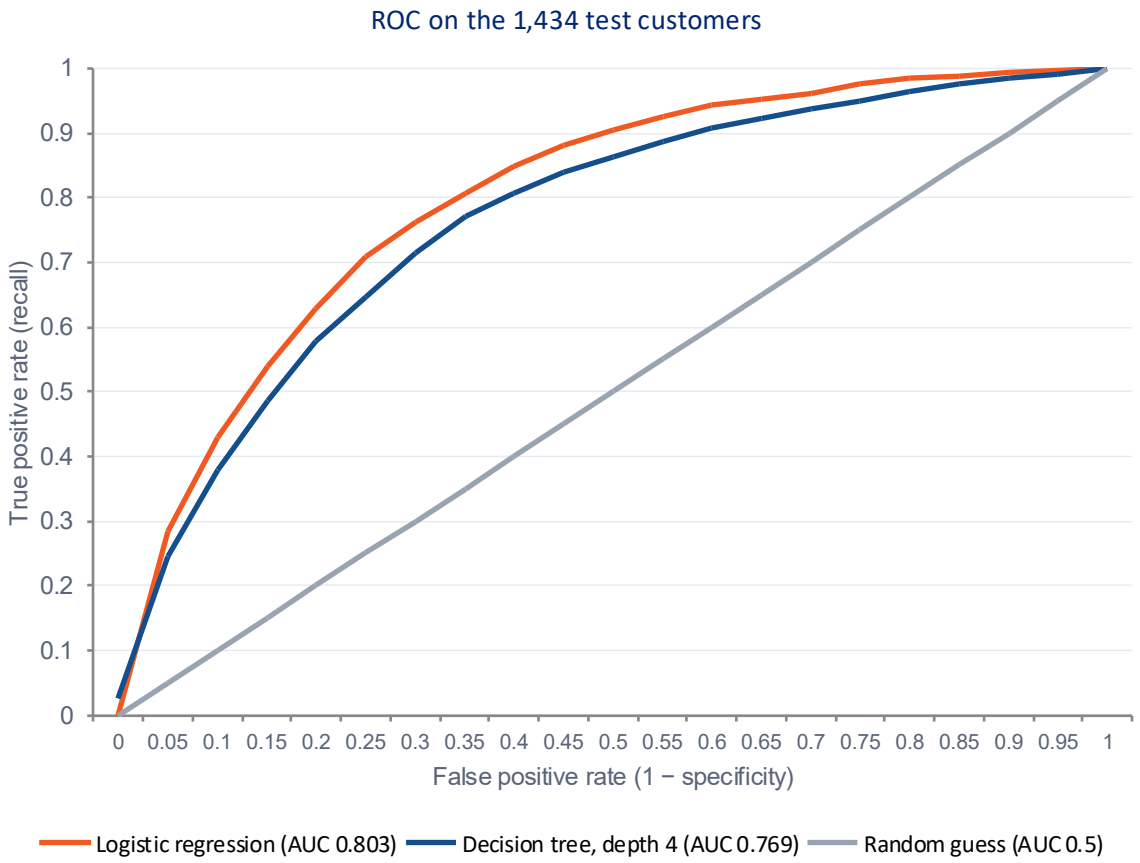
$\bar{d} = 0.0232$, $s_d = 0.0110$, $t = 0.0232/(0.0110/\sqrt{10}) = 6.69$

6.69 > 2.262: reject H_0. Logistic regression is significantly more accurate than the depth-4 tree on this file

With noisier folds (s_d = 0.04), t would be 1.8 and the same 2.3-point gap would count as chance. Always report the folds as well as the means.

4.6 Model Selection

ROC curves and AUC: comparing classifiers across every threshold



Each point on a curve is one probability threshold; the closer to the top-left corner, the better

How a ROC curve is drawn

- Sort test tuples by predicted P(returns), from high to low
- Lower the threshold one tuple at a time: a true returner moves the curve up, a non-returner moves it right
- AUC = area under the curve = probability that a random returner is ranked above a random non-returner

Choosing a model is more than accuracy

- Speed: to train, and to classify one customer (k-NN is slow here)
- Robustness to noise and missing values; scalability to millions of rows
- Interpretability: can the decision be explained to the customer, the regulator, the board?
- Cost of errors: AUC ignores it; pick the threshold where expected cost is lowest

4.6 Improving Accuracy

Ensemble methods: a committee of models beats one expert

Bagging

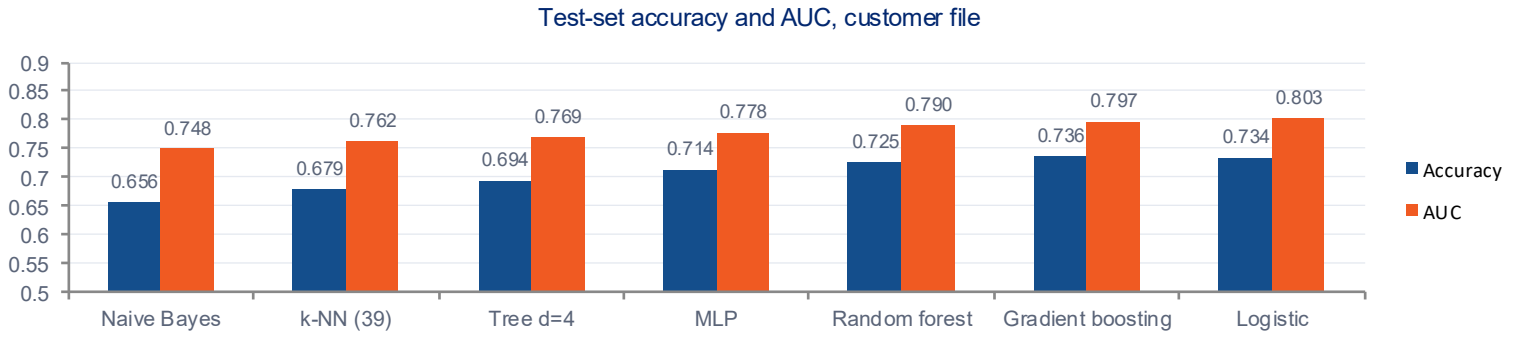
- Draw k bootstrap samples, train one classifier on each, let them vote
- Reduces variance: the wild tree grown to purity becomes stable in a forest
- Random forest = bagged trees + a random subset of features at every split

Boosting

- Train models in sequence; re-weight the tuples so the next model works harder on the misclassified ones
- Final vote weighted by each model's accuracy (AdaBoost); gradient boosting fits residuals
- Reduces bias; can overfit noisy data; XGBoost and LightGBM in industry

Stacking

- Train different algorithms (tree, Bayes, k-NN, logistic)
- A meta-learner learns how to combine their predictions
- Wins competitions; harder to explain and to maintain



On this file the ensembles beat the single tree by 3 to 4 points and land next to logistic regression. Clean, near-linear data rewards simple models; ensembles shine when interactions are strong.

4.6 Improving Accuracy

AdaBoost step by step, and random forests

1. Start with equal tuple weights 1/d for the d training tuples
2. Round i: sample D_i with replacement, each tuple's chance proportional to its weight; train classifier M_i on D_i
3. Compute the error of M_i on D_i: error(M_i) = Σ w_j · err(X_j), the sum of the weights of misclassified tuples
4. If error(M_i) > 0.5, discard M_i and resample. Otherwise multiply the weight of each correctly classified tuple by error(M_i)/(1 − error(M_i)) and normalise
5. Repeat for k rounds. To classify X: each M_i votes with weight log((1 − error(M_i))/error(M_i)); the class with the largest total wins

Why boosting works, and its risk

- Each round concentrates on the tuples the committee still gets wrong: bias falls
- Weak learners (stumps, shallow trees) are enough; the sequence makes them strong
- Risk: it also chases mislabelled tuples, so noisy data can be overfitted

Random forest (Breiman 2001)

- Each classifier is a decision tree grown on a bootstrap sample
- At every node only a random subset of F attributes is considered (Forest-RI); or random linear combinations (Forest-RC)
- Trees vote; accuracy comparable to AdaBoost, more robust to noise and outliers, fast, and it gives feature importances (lab task 5)

4.6 Improving Accuracy

Class imbalance, threshold moving, and tuning the knobs

Sampling the training set

- Oversampling: repeat or synthesise minority tuples (SMOTE interpolates between neighbours)
- Undersampling: drop majority tuples until balanced
- Both change only the training set; the test set keeps the real rates

Cost-sensitive learning

- Give a misclassified minority tuple a higher cost: class_weight='balanced' in scikit-learn
- Cost matrix in the decision: minimise expected cost instead of error count
- Ensembles of the minority class with subsets of the majority

Threshold moving

- Keep the model, lower the cut-off on P(positive) from 0.5 to 0.3: recall up, precision down
- Read the trade-off from the ROC or precision-recall curve
- Cheapest fix, and the one the business can set itself

Features: garbage in, garbage out

- Select: drop attributes with no predictive value (χ^2, correlation, tree importance)
- Construct: recency, frequency, monetary were built from raw transactions; that step gave the tree its root

Hyper-parameter tuning

- Grid search: try every combination of max_depth, min_samples_leaf, k, C, learning rate
- Random search: sample combinations; always score by cross-validation, never on the test set

Practice 6

1. Logistic regression on the same 1,434 test customers gave TP = 471, FN = 198, FP = 184, TN = 581. Compute accuracy, precision, recall, specificity and F1. Compare each with the tree. Which model would you use for a retention campaign with a fixed budget of 400 vouchers?

2. A credit-card issuer has 200,000 transactions with 1,000 frauds. Model A flags 2,000 transactions and catches 800 frauds. Model B flags 200 and catches 150. Compute precision and recall for both, then argue which is better if an investigator can review 300 alerts per day.

3. You have 350 loan files. Your colleague proposes a single 70/30 split and reports 81% accuracy. Propose a better evaluation plan, say what number you would report instead, and explain why you would run the plan with class stratification.

Quick check

1. Which measure does ID3 use to choose the split attribute?

A. Gini index
B. Gain ratio
C. Information gain
D. χ^2

2. Naive Bayes is 'naive' because it assumes that

A. all classes are equally likely
B. attributes are independent given the class
C. attributes are Gaussian
D. the training set is large

3. In backpropagation, the error of a hidden unit depends on

A. only its own output
B. the true label directly
C. the errors of the units it feeds, weighted
D. the learning rate

4. A tree with 100% training and 64% test accuracy is

A. well fitted
B. underfitted
C. unbiased
D. overfitted

5. Fraud is 0.5% of transactions. The metric to watch is

A. accuracy
B. recall of the fraud class
C. specificity
D. error rate

6. 10-fold cross-validation tests each tuple

A. ten times
B. never
C. exactly once
D. 63.2% of the time

Lab 4 in Google Colab and submission

Task	What to produce
Task 1	Load saigon_bean_customers.csv, encode the two categorical columns, split 70/30 with stratification, and report the class balance in both parts
Task 2	Grow decision trees of depth 2, 4 and unlimited; print the depth-3 tree as text; tabulate training vs test accuracy and explain the gap
Task 3	Fit Gaussian naive Bayes, k-NN (tune k from 1 to 40 on a validation curve), logistic regression and an MLP; collect confusion matrices
Task 4	Compute accuracy, precision, recall, F1 and AUC for every model, draw the ROC curves on one chart, and run stratified 10-fold cross-validation
Task 5	Train a random forest and a gradient boosting model, compare with task 4, and plot feature importances; then move the threshold to reach recall ≥ 0.80
Task 6	Fit the multiple regression for sept_spend_vnd on returning customers, report R^2 and RMSE on a hold-out, and write five sentences for the marketing manager

SUBMISSION

File named StudentID_FullName_DM_Lab04.ipynb on the LMS, plus the Colab share link, before Practical Session 2. Run all cells before you submit. The notebook reproduces every number quoted in this lecture, so use it to check your hand computations from Practices 1 to 6. Group report (CLO5): one page on tasks 5 and 6, written for a manager who has not taken this course.

Summary

1. Classification predicts a category, numeric prediction predicts a number. Both learn from labelled data in two steps, and accuracy is measured on unseen data.
2. A decision tree splits top-down by information gain, gain ratio or Gini. On the credit table age wins with gain 0.246. Prune it, or it memorises the training set.
3. Naive Bayes multiplies class-conditional probabilities under an independence assumption. The applicant (youth, medium, collateral yes, fair) is approved with 0.028 against 0.007.
4. A neural network chains logistic units. Backpropagation pushes each weight against the error gradient, one epoch at a time. The result is accurate but hard to explain.
5. Rules read as policy, whether they come from a tree, from sequential covering or from association rules. k-NN classifies by distance, SVM by the widest margin.
6. Regression predicts a number by least squares. The September spend model explains 80% of the variance with four features.
7. The confusion matrix gives accuracy, precision, recall, specificity and F1. Choose the metric from the cost of each error, and estimate it by stratified k-fold cross-validation.
8. ROC and AUC compare models across thresholds. Ensembles, resampling, cost-sensitive learning, threshold moving and tuning are the levers for improving accuracy.

Data for this chapter

File	Rows	One row is	Columns
credit_applicants.csv	14	One loan application	age, income, collateral, credit_rating, loan_approved (label)
saigon_bean_customers.csv	4,777	One identified customer, June to August 2026	main_store_type, main_channel, has_loyalty_card, frequency, avg_basket_vnd, monetary_vnd, recency_days, weekday_share, morning_share, discount_share, distance_km, returned_sept (label), sept_spend_vnd (numeric target)

The credit table: for the blackboard

- Four categorical attributes, one yes/no label, 9 approved and 5 refused
- Same structure as the AllElectronics table in the textbook, renamed to a bank
- Every entropy, gain, Gini and Bayes number on the slides is computed on it

The customer file: for Colab

- Built from saigon_bean_transactions.csv: one row per customer_id with RFM features
- Label returned_sept = 1 if the customer bought again in September (47% did)
- Numeric target sept_spend_vnd for the regression part of section 4.5

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KINH TẾ - LUẬT

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF ECONOMICS AND LAW

THANK YOU!

Next: Chapter 5, Clustering

